

ReTool: 用强化学习教大模型“会用工具”—— 在推理中策略性地调用代码解释器

Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin,
Baoquan Zhong, Chengquan Jiang, Jinxin Chi, Wanjun Zhong
ByteDance Seed (字节跳动 Seed 团队)

英文原标题: ReTool: Reinforcement Learning for Strategic Tool Use in LLMs

发表: arXiv:2504.11536v2, 2025 年 4 月 | **引用数:** 297 (Semantic Scholar, 2026-06-07)

代码 / 项目主页: <https://github.com/ReTool-RL/ReTool> <https://retool-rl.github.io/>

一句话定位: 这篇论文要解决的是——“会推理”的大模型为什么算不准数、解不对复杂方程。它的答案是：用强化学习 (RL) 让模型自己学会在文字推理过程中“随手写一段代码 → 真跑一下 → 把运行结果接着往下想”，而“什么时候该写代码、写什么代码”全靠 RL 摸索出来，不靠人写死规则。

1 读这篇前要懂的几个概念 (小白知识卡)

下面这些是读懂这篇论文的“前置知识”。如果你是 AI 应用开发的新手，先花两分钟扫一遍。

1. 大语言模型 (LLM) 与思维链 (CoT)

大语言模型 (large language model, LLM) 就是 GPT、DeepSeek 这类会“说话”的模型。**思维链** (chain-of-thought, CoT) 指的是：让模型在给最终答案前，先用自然语言一步步“把思路写出来”，就像做数学题写草稿。写草稿往往能让模型答得更准。本文研究的“推理模型”（如 OpenAI o1、DeepSeek R1）就是特别擅长写长草稿、还能自我检查的模型。

2. 强化学习 (RL) 与 PPO

强化学习 (reinforcement learning, RL) 是一种“通过试错 + 奖惩”来训练模型的范式：模型先生成一批答案（这一步叫 **rollout**，可以理解为“让模型自由发挥跑一遍”），然后根据答案好坏给一个**奖励** (reward)，好的行为被强化、坏的被抑制。**PPO** (Proximal Policy Optimization, 近端策略优化) 是目前最常用的 RL 算法之一，它的核心技巧是：每次更新不让模型“变化太大”（用一个 clip 裁剪机制限制步子），从而训练更稳。

3. RLVR：用“可验证的结果”当奖励

本文用的是**结果奖励** (outcome reward)：不管模型中间过程怎么折腾，只看最终答案对不对，对了给 +1、错了给 -1。因为数学题答案唯一、可以程序自动判分，所以这类奖励叫“可验证奖励”（这一思路常被称为 RLVR, RL with Verifiable Rewards）。好处是**不需要人工标注**每一步，也不容易被模型“钻空子刷分” (reward hacking)。

4. 工具集成推理（TIR）与代码解释器（CI）

代码解释器 (code interpreter, CI) 就是一个能真正运行 Python 代码的“沙箱” (sandbox, 隔离的安全运行环境)。**工具集成推理** (tool-integrated reasoning, TIR) 指的是：让模型在推理时不要光靠脑子硬算，而是把“枚举、验证、精确计算”这种活儿交给代码去做。本文就是把代码解释器接进了模型的推理回路里。相关的早期思路还有 PoT (Program-of-Thought, 程序化思维)、PAL (程序辅助语言模型)。

5. 冷启动 SFT (cold-start supervised fine-tuning)

RL 训练前，模型最好已经“大致会”做这件事，否则一上来全是随机乱试、学不动。所以本文先用一批高质量示范数据做**监督微调** (supervised fine-tuning, SFT)，教模型“代码该长什么样、什么时候插进来”，给后续 RL 一个好的起点——这一步就叫**冷启动** (cold-start)。

2 摘要（全文翻译）

用强化学习 (RL) 训练的推理模型（例如 DeepSeek R1）在文字推理上表现出色，但在需要结构化求解的场景里却力不从心——比如几何推理、需要简洁计算的题目、或复杂方程求解；而这些恰恰是代码解释器 (code interpreter, CI) 这类计算工具的强项。为弥合这一鸿沟，我们提出 **ReTool**，用“工具集成学习”增强长链推理，它有两个关键特性：(1) 在自然语言推理过程中**动态地交错插入实时代码执行**；(2) 一套**自动化的 RL 范式**，允许策略在 rollout 中进行多轮、实时的代码执行，并仅凭结果反馈教模型学会“何时、如何调用工具”。

ReTool 采用一套系统化的训练框架：先用合成的冷启动数据，生成“带代码的长链推理轨迹”，用来微调基座模型；随后的 RL 训练以任务结果作为奖励，迭代地打磨模型的工具使用策略，使其能够**自主发现最优的工具调用模式，而无需人类先验**。在具有挑战性的数学奥赛基准 AIME 上的实验证明了 ReTool 的优越性：我们的 32B 模型仅用 **400 个训练步**就在 AIME2024 上达到 **67% 的准确率**，无论在效率还是性能上都超过了文字 RL 基线（40% 准确率，1080 步）。更值得注意的是，在扩展设置下 ReTool-32B 取得了 **72.5% 的准确率**，比 OpenAI 的 o1-preview 高出 **27.9%**。进一步分析揭示出一些**涌现行为** (emergent behaviors)，例如代码自我纠错，标志着模型自主掌握自适应工具使用的“顿悟时刻” (aha moment)。这些发现凸显了“结果驱动的工具集成”在推进复杂数学推理上的潜力，也为混合的“神经—符号”系统 (neuro-symbolic systems) 提供了新见解。

日期：2025 年 4 月 15 日 项目主页：<https://retool-rl.github.io/>

3 逐节精读

3.1 第 1 节引言：为什么“会推理”还不够？

作者先承认现状：RL 已经成为提升 LLM 推理能力的主流范式，o1、DeepSeek R1 这些模型靠它学会了自我纠错、更审慎地思考，甚至显露出“元认知”（metacognition，即不仅会想，还会监督和修正自己的思考）的苗头。

但有个老大难：纯文字推理算不准数。几何题、需要精确数值计算的题、复杂方程，这些场景下模型光靠“语言模式”硬算，特别容易出小错，而且错误会层层累积（论文叫 compounding error，复合误差）。相比之下，代码解释器提供的是一个形式化、可执行的接口，擅长枚举、验证、精确计算，还能让模型通过“写程序去探索”来扩大解题的搜索空间。

那为什么不直接教模型用工具呢？之前的做法主要是提示工程和监督微调（SFT）。问题在于：SFT 只是在模仿人精心准备的那批数据，学到的是固定套路，一旦题目变样就不灵了，模型可能乱用工具或退化成脆弱的经验法则。作者的洞察是：**RL 才是正解**——它让模型自由探索推理轨迹，靠“结果好不好”这个反馈来学策略，从而能学会一些精细的行为，比如“代码跑错了怎么从错误中恢复”“长推理里到底该在哪一步调用代码”。

ReTool 的两大组件（这是全文骨架，务必记住）：

1. **冷启动数据构造**：做一个高质量数据集，明确示范“何时、如何调用代码解释器”，让模型先具备基本的工具使用与“看懂运行结果”的能力。
2. **工具增强的 RL**：在此基础上用 RL 训练，让模型自己发现最优的工具操控策略，并通过结果奖励不断调整行为——这是 SFT 单独做不到的。在长链推理中，策略模型会灵活地写代码块、从沙箱式代码解释器拿到实时运行结果，再据此继续往下想。

引言里抢先剧透的结果：基于 Qwen2.5-32B-Instruct，ReTool 仅 400 步就在 AIME2024 上达 67.0%，而文字 RL 基线跑 1080 步才 40.0%。换更强的基座 DeepSeek-R1-Distill-Qwen-32B 后还能更高，超过 QwQ-32B-Preview、s1-32B、o1-preview 等强基线。另外，光是冷启动模型（还没 RL）就有 40.9%，已经和文字 RL 基线打平、远超原始 Qwen2.5-32B-Instruct 的 26.7%——说明数据集本身就有效地捕捉到了工具使用模式。

3.2 第 2 节方法：ReTool 是怎么做的

3.2.1 2.1 总览：两个阶段

整个方法分两步走：(1) **冷启动 SFT**（给 RL 一个稳健的初始化）；(2) 带“**交错代码执行**”的 **RL**（专门训练模型在推理中恰当地选择和使用工具）。

3.2.2 2.2 冷启动：打好工具集成推理的底子

做法是设计一条数据流水线：

- **先攒纯文字推理数据**：从多个来源（含开源的 OpenThoughts 等）收集数学推理数据，再用“人审 + DeepSeek-R1 评估”双重验证过滤掉无效样本，得到高质量文字数据集，记作 D_{init} 。
- **再自动转成“带代码”的数据**：用一个结构化提示模板，把原推理过程里那些适合用代码做的手工计算步骤，替换成对应的代码片段 + 解释器运行结果。然后做两阶段验证：第一阶段查**格式**（保证语法一致、可读，便于后续 RL 里识别“何时该触发工具调用”）；第二阶段查**答案**（最终答案不对的样本直接扔掉）。最后得到带代码的长链推理数据集 D_{CI} 。
- 在 D_{CI} 上做 **SFT**，让模型学会“何时、如何调用代码解释器”。

白话总结：先用大模型把“人脑硬算的步骤”改写成“写代码算”，再严格筛掉算错的，拿这批干净示范喂给模型打底。

3.2.3 2.3 ReTool 的 RL：策略性工具使用

训练算法 (PPO)。 ReTool 基于 PPO 算法，优化目标如下（公式 1）：

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{(q,a) \sim D, o_{\leq t} \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\min \left(\frac{\pi_{\theta}(o_t|q, o_{<t}; \text{CI})}{\pi_{\theta_{\text{old}}}(o_t|q, o_{<t}; \text{CI})} \hat{A}_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t}; \text{CI})}{\pi_{\theta_{\text{old}}}(o_t|q, o_{<t}; \text{CI})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_t \right) \right]$$

逐符号拆解（看懂这一个公式就懂 PPO 了）：

- π_{θ} ：当前正在训练的**策略模型**（policy model）， θ 是它的参数； $\pi_{\theta_{\text{old}}}$ 是更新前的**旧策略**（参考模型）。
- q 是问题， a 是标准答案， D 是数据集； o_t 是模型在第 t 步生成的 token， $o_{<t}$ 是它前面已生成的内容。
- **关键点** $\pi_{\theta}(o_t|q, o_{<t}; \text{CI})$ ：这里多了一个“; CI”，意思是生成过程中夹带了代码解释器的实时执行与反馈——这正是 ReTool 对标准 PPO 的改造：rollout 不再是纯文字，而是“文字 + 真跑代码”。
- $\frac{\pi_{\theta}(\cdot)}{\pi_{\theta_{\text{old}}}(\cdot)}$ 是**重要性比率**（新旧策略对同一动作的概率之比）； $\hat{A}_t$ 是**优势函数**（advantage，衡量“这一步比平均水平好多少”）。
- $\text{clip}(\cdot, 1 - \varepsilon, 1 + \varepsilon)$ 把这个比率**裁剪**在 $[1 - \varepsilon, 1 + \varepsilon]$ 区间内，再取 min。这就是 PPO 的“别一步迈太大”的安全阀，保证训练稳定。

奖励设计 (公式 2)。为了教模型“何时、如何用工具”，作者用了一个极简的基于规则的准确率奖励：要求模型把最终答案放进指定格式（如 `\boxed{\}`）里，便于程序自动判分。奖励定义为：

$$R(a, \hat{a}) = \begin{cases} 1, & \text{若 is_equivalent}(a, \hat{a}) \text{ (预测答案与标准答案等价)} \\ -1, & \text{否则} \end{cases}$$

其中 a 是标准答案、 $\hat{a}$ 是模型预测答案。**为什么这么简单？**作者特意只看最终对错、不给“代码能否运行”单独加分，就是为了**避免 reward hacking (钻奖励空子)**、并鼓励模型探索更多样的解题行为。这也呼应了 RLVR 的思路：奖励越简单越纯粹，越不容易被带偏。

带交错代码执行的 Rollout (方法的灵魂)。 这是 ReTool 跟普通 RL 最不同的地方 (见后文图 1)。普通 rollout 只生成文字；ReTool 的 rollout 让策略模型和外部代码沙箱协作，产出“文字 + 代码 + 实时解释器反馈”的混合内容。具体流程是：

1. 模型用 `<code></code>` 标签把要执行的代码圈出来；它先正常生成文字推理 t_1 。
2. 一旦检测到代码结束标记 `</code>`，**生成就暂停**，把其中的代码 c_1 解析出来、送进代码沙箱去跑。
3. 沙箱跑完后，把输出 f_1 (成功结果或报错信息) 填进 `<interpreter></interpreter>` 标签，回灌给模型。
4. 模型接着往下生成，直到给出最终答案 o 、或者再写一段新代码。最终形成一条混合推理轨迹 $[t_1 \oplus c_1 \oplus f_1 \oplus \dots \oplus o]$ ($\oplus$ 表示拼接)。

划重点：报错信息也会原样回灌给模型。正是这个“动态反馈”机制，让模型能从错误中学习、迭代修正代码——这是后面“代码自我纠错”涌现行为的来源。

2.3.2 训练细节 (工程上的几个关键点)。

- **框架与超参：**用 VeRL 框架；PPO；冷启动数据训 2 个 epoch；AdamW 优化器，初始学习率 $1e-6$ ；最大序列长度 16384 token；mini-batch size 512；**KL 系数设为 0.0** (即不加 KL 约束，让模型放开探索)；主基座 Qwen2.5-32B-Instruct。
- **解释器反馈掩码 (Interpreter Feedback Mask)：**把 `<interpreter>` 里的反馈内容从损失计算中屏蔽掉。因为这些是外部塞进来的 token、不是模型自己生成的，算进 loss 会干扰训练、破坏模型自己的连贯推理。这是个很重要的稳定性技巧。
- **KV-Cache 复用：**每次触发 `</code>` 时，把代码执行前的 KV-cache 全缓存下来，之后只计算并追加解释器反馈那段，大幅降低显存开销。
- **异步沙箱：**沙箱以“worker 池”方式运行，各 worker 按自身负载主动拉任务，做到负载均衡、并行执行，避免慢线程拖后腿，保证训练吞吐稳定。

3.3 第 3 节实验：见后文“实验解读”

(这一节的数据集、基线、主结果与“认知行为分析”，我们集中放在第 5 节展开讲。)

3.4 第 4 节背景与相关工作

- **4.1 LLM 推理**: CoT 由 Wei 等人首创, 用自然语言一步步描述来增强推理。研究重心已从“训练时扩展”转向“**测试时扩展**”(test-time scaling, 即推理时多花算力换更好结果), 手段包括逐步偏好优化 (Step-DPO)、蒙特卡洛树搜索 (MCTS)、RL 等。与 CoT 互补的是 **PoT** (Program-of-Thought) 和 PAL, 它们把 Python 解释器等外部工具接进来简化与验证复杂步骤。
- **4.2 工具集成推理 (TIR)**: TIR 最早是为帮 LLM 解决计算密集的数学题而引入的。Wang 等人提出文字推理与代码执行相互验证的迭代方法; Chen 等人用 SFT 在自建的“带代码 CoT”数据上集成代码执行——但 SFT 受限于特定数据分布, 学不到“何时、如何调用工具”这种自适应策略。有一项同期工作把 RL 用在 1.5B/7B 的 Qwen2.5-Math 上学工具使用, 但效果还不够好。**ReTool** 把这条路线放大到了 **32B**, 并显著超过 Qwen-Math-72B-TIR 和 o1-preview。

3.5 第 5 节结论

本文提出 ReTool: 一个让 LLM 通过高效使用代码解释器来自**我增强数学推理**的 RL 框架。在 AIME2024/2025 上的实验表明, ReTool 不仅准确率超过传统文字 RL, 而且**收敛所需训练步数大幅减少**。通过精心的数据构造和专门的工具使用流水线, ReTool 让模型发展出成熟的“计算介入”策略, 为更高效、更强大的工具增强推理铺平了道路。

4 方法图解：一张图看懂 ReTool 的训练回路

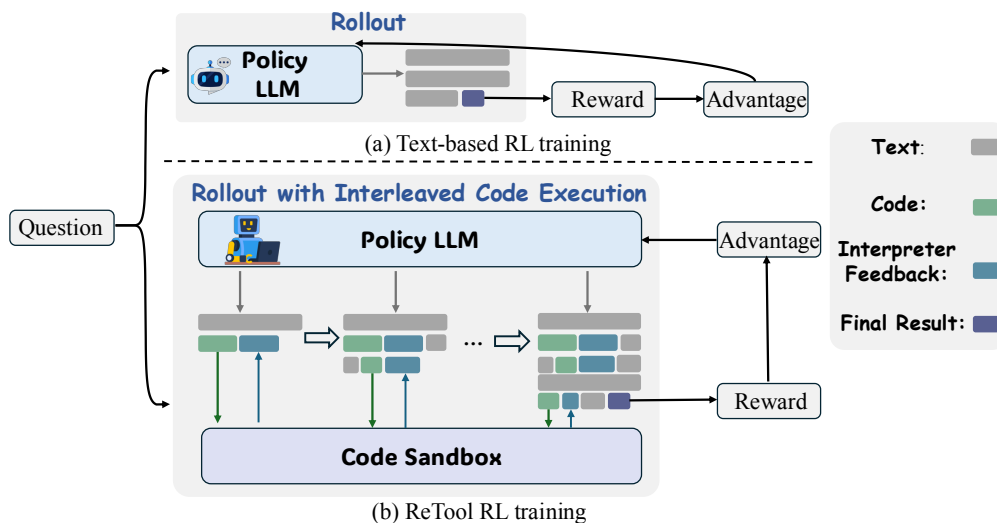


Figure 2 Demonstration of text-based RL training process and ReTool's RL training process.

图 1: 文字 RL 训练流程与 ReTool RL 训练流程的对比 (原文图 2)。上半 (a) 是普通文字 RL; 下半 (b) 是 ReTool。图例: Text= 文字、Code= 代码、Interpreter Feedback= 解释器反馈、Final Result= 最终结果。

这张图在说什么、数据怎么流动？

- 上半 (a) 文字 RL (对照组): 问题 (Question) 进 *Policy LLM* (策略模型), 它一口气吐出一长串纯文字 (图中灰色块 = Text), 到末尾产出最终结果 (深蓝小块) → 算 *Reward* (奖励) → 转成 *Advantage* (优势) 回传更新模型。全程没有代码、没有外部执行。
- 下半 (b) ReTool (本文方法): 同样从 Question 出发进 *Policy LLM*, 但生成过程是“文字块 (灰) + 代码块 (绿) + 解释器反馈块 (蓝)”交错进行的。每当模型写完一段代码 (绿色), 就往下打到 **Code Sandbox (代码沙箱)** 里真跑一遍, 沙箱把运行结果 (蓝色) 回灌给 *Policy LLM*; 模型据此继续往下想、可能再写新代码……如此循环 (图中 “...” 表示多轮), 直到产出最终结果 (深蓝) → 同样走 *Reward* → *Advantage* 更新模型。
- 一句话抓住本质: (a) 是“闷头写作文”, (b) 是“边写边查、写一段就跑一段、跑错了改了再写”。而“在哪一步写代码、写什么代码”不是人规定的, 是模型在 RL 里靠最终对错的奖励自己摸索出来的。

5 实验解读：这些数字说明了什么

评测设置。 在 **AIME2024 & AIME2025**（美国数学邀请赛，高难数学竞赛题）上评测。为了评测稳定，每个评测集 **重复跑 32 遍取平均**来估计 pass@1（“一次答对”的准确率）。推理超参：temperature 1.0、top-p 0.7。对比的强基线包括 Qwen2.5-Math-72B-Instruct(-TIR)、Sky-T1、DeepSeek-R1-Zero-Qwen-32B、QwQ-32B-Preview、s1-32B、OpenAI o1-preview，以及最关键的对照——**不带工具的文字 RL**（Text-based RL，同样基于 Qwen2.5-32B-Instruct）。

表 1：主结果（原文表 1，AIME pass@1，单位%）。♠ 文字 RL 也包含了文字版冷启动 SFT 以保证公平对比；◇ 冷启动模型在推理时同样接入了代码解释器。“—” 表示原文未报告。

模型	AIME2024	AIME2025
<i>已有基线 (Existing Baselines)</i>		
Qwen2.5-Math-72B-Instruct	30.0	—
Qwen2.5-Math-72B-Instruct-TIR	40.0	—
Sky-T1	43.3	—
OpenAI o1-preview	44.6	37.9
DeepSeek-R1-Zero-Qwen-32B	47.0	—
QwQ-32B-Preview	50.0	33.5
s1-32B	56.7	—
<i>CI-powered RL (本文方法)</i>		
ReTool (Qwen2.5-32B-Instruct)	67.0	49.3
ReTool (DeepSeek-R1-Distill-Qwen-32B)	72.5	54.3
<i>消融 (基于 Qwen2.5-32B-Instruct)</i>		
不训练 (Base Model)	26.7	—
去掉 CI (文字 RL ♠)	40.0	36.7
去掉 RL (仅冷启动 ◇)	40.9	34.5

主结果解读。

- **又准又快：** ReTool(Qwen2.5-32B) 仅 **400 步**就到 67.0% / 49.3%，而文字 RL 跑 1000+ 步才 40.0% / 36.7%。这说明“把工具使用显式纳入决策过程”不光提升了上限，还大幅提升了训练效率。
- **超过强基线：** AIME2024 上比 s1-32B 高 10.3 个百分点；AIME2025 上比 o1-preview 高 11.4 个百分点。换更强基座 DeepSeek-R1-Distill-Qwen-32B 后进一步涨到 72.5% / 54.3%。
- **消融最有信息量：** 把 ReTool 拆开看——光冷启动（40.9%）就已追平文字 *RL*（40.0%）、远超不训练的 26.7%；说明**数据集本身有效**。而真正把成绩从 40% 拉到 67% 的，是在**带工具的 rollout 上做 RL** 这一步。即：冷启动给底子，*RL* 才是质变。

认知行为分析（第 3.3 节，本文最有意思的部分）。 作者在 RL 训练的各个 checkpoint 上统计了一系列“代码相关行为”指标（原文图 3），发现模型在 RL 过程中自发变强：

- **回答更短、更高效**：平均回答长度先骤降后缓升，最终比 RL 前**短约 40%**（从约 10k 降到约 6k token）。原因是“用一段代码替换掉冗长的手算过程”，反而省了 token。
- **越来越爱用代码**：含代码的回答比例（code ratio）一路上升，最终**近 98%** 的题都会用到代码。
- **代码越写越复杂**：平均代码行数持续上升，训练结束时约为 RL 前的 **5 倍**；测试集“正确代码”总数从 1k 升到 5k。
- **越用越对**：正确回答里的代码通过率始终接近 **100%**；错误回答里的代码通过率则呈下降趋势——说明“代码能不能跑通”确实影响最终对错。
- **调用时机提前**：代码调用时机（代码起始位置 / 回答总长）在训练中不断提前，说明模型学会了该早用就早用。
- **“顿悟时刻”——代码自我纠错**（原文图 4）：在没有任何“纠错示范数据”的情况下，模型自发学会了改 bug。典型案例：模型先写的代码因函数 `greedy()` 未定义而报 `NameError`；收到报错后，模型反思 *“Oops, the functions need to be defined in the same scope. Let’s correct that.”*（哎呀，函数得定义在同一作用域里，我改一下），随后补全定义、生成了能跑的版本。这是 RL 培养出“元认知”的有力证据。
- **代码用途更多元**（图 5）：RL 前代码主要用于“计算 + 验证”；RL 后用途变得更丰富多样，体现出“自适应选工具”的能力，也增强了泛化性。
- **对比案例**（图 6）：同一道题，纯文字推理靠“费力的手算”、容易算错而答错；ReTool 则用简洁代码替代复杂计算，既保证算得对，又让模型能把精力放在整体解题策略上。

6 核心 takeaways 与对 AI 应用开发的启示

1. **“会用工具”可以被 RL 教出来，而且不靠人定规则。** ReTool 最核心的贡献是证明了：模型“何时/如何调用代码”这种策略，完全可以仅凭最终结果的奖励自学成才，比人工写规则或纯 SFT 模仿更鲁棒、更能泛化。

2. **把“能验证的活儿”交给工具，把“想清楚”留给模型。** 精确计算、枚举、验证这类事，代码做得又快又准；模型负责高层推理与决策。这种“神经 + 符号”的分工，对所有需要精确性的 AI 应用（数据分析、财务计算、代码辅助、科学计算）都是可直接借鉴的范式。

3. **奖励越简单越好，警惕 reward hacking。** 作者特意只奖励最终答案对错、不给“代码能跑”单独加分，就是怕模型钻空子（比如为了拿“可执行”分而乱写代码）。做 RLVR 时，**选一个干净、可自动验证、难以作弊的奖励信号**往往比堆砌复杂奖励更重要。

4. **工程细节决定成败：反馈掩码 + KV-Cache 复用 + 异步沙箱。**想自己搭“带真实工具执行的 RL”，这三点是关键：把外部塞进来的 *token* 从 *loss* 里屏蔽（否则训练不稳）、复用代码执行前的 *KV-cache*（省显存）、异步并行的沙箱池（别让代码执行拖慢训练）。这些是论文里很实在的可复用经验。

5. **冷启动给底子，RL 给质变。**消融实验清楚地告诉我们：好的示范数据（冷启动 SFT）能让模型“及格”，但要从 40% 跨到 67%，必须靠在真实工具交互上做 *RL*。做 agent / 工具调用产品时，别指望只靠造数据微调就到顶，真正拉开差距的是“让模型在带真实反馈的环境里试错学习”。

6. **“顿悟时刻”是涌现的。**代码自我纠错没有被专门教过，却在 RL 中自发出现。这提示我们：只要给对环境（能拿到真实报错反馈）+ 奖励（结果导向），模型能涌现出超出训练数据的高级行为。把“真实的失败反馈”喂回给模型，是培养这种能力的关键。

参考文献（仅列最关键几条，英文原样）

1. DeepSeek-AI et al. *DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning*, 2025. <https://arxiv.org/abs/2501.12948>
2. John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov. *Proximal Policy Optimization Algorithms*, 2017. <https://arxiv.org/abs/1707.06347>
3. Wenhua Chen, Xueguang Ma, Xinyi Wang, William W. Cohen. *Program of Thoughts Prompting: Disentangling computation from reasoning for numerical reasoning tasks*, 2023. <https://arxiv.org/abs/2211.12588>
4. Xuefeng Li, Haoyang Zou, Pengfei Liu. *ToRL: Scaling tool-integrated RL*, 2025. <https://arxiv.org/abs/2503.23383>
5. Qwen et al. *Qwen2.5 Technical Report*, 2025. <https://arxiv.org/abs/2412.15115>
6. OpenAI. *Learning to reason with LLMs*, 2024. <https://openai.com/index/learning-to-reason-with-llms>